

# Point-by-Point Response to Reviewer Comments

Manuscript: “aValia+Tec: A Tamper-Evident Platform for  
Integrity-Verified Academic Selection Processes”

L.D.V. Santos, V.R.Q. Leithardt

March 4, 2026

Dear Editor and Reviewers,

We sincerely thank the Editor and both Reviewers for their thorough, constructive, and insightful evaluation of our manuscript. Their comments identified important weaknesses that have substantially improved the scientific rigor, transparency, and honesty of the paper.

We have addressed every comment raised by both reviewers. Below we provide our detailed point-by-point response.

## Reviewer #1

### Comment 1.1 — Threat model and “immutable” claim

#### Reviewer Comment

The threat model contains a critical vulnerability that the authors themselves acknowledge: a server administrator with root-level database access could rewrite the entire hash chain consistently. This fundamentally undermines the paper’s central claim of an “immutable” audit trail. Without external hash anchoring (e.g., a public blockchain or an RFC 3161-compliant timestamping authority), the use of the term “immutable” is technically incorrect and misleading. The authors must either implement this safeguard or substantially revise their claims.

**Response:** We fully agree with this criticism. The term “immutable” was technically inaccurate in the absence of external anchoring, and we thank the reviewer for pressing this point.

In the revised version, we have taken both corrective actions the reviewer suggested:

1. Implementation of RFC 3161 Trusted Timestamping. We implemented a `TimestampService` module that submits each submission’s SHA-256 digest to an external RFC 3161-compliant Timestamp Authority (default: FreeTSA.org). The TSA returns a cryptographically signed `TimeStampResp` (TSR) token that is stored alongside the submission record. Even a root-level database administrator cannot forge a valid TSR without the TSA’s private signing key. The code includes ASN.1 DER encoding, HTTP POST to the TSA, and TSR verification logic. A corresponding SQL migration (`002_add_rfc3161_timestamp.sql`) adds `tsr_base64`, `tsr_tsa_url`, and `tsr_requested_at` columns. The `/api/verify/:protocol` route now returns TSR verification results alongside the existing hash check.
2. Terminology correction. Every occurrence of “immutable” has been replaced with “tamper-evident” throughout the manuscript, the abstract, highlights, the architecture diagram label, and all technical descriptions.

**Changes in manuscript:** *Abstract rewritten; Section 1 motivation; Section 2 Software Functionalities — new RFC 3161 description paragraph; Section 3 Threat Model — completely rewritten with three adversary classes (external attacker, non-privileged insider, privileged administrator) and residual-risk analysis; architecture diagram: “Immutable Event Store” → “Tamper-Evident Store”; Section 7 Conclusions — tamper-evident language. New code files: `TimestampService.js`, `002_add_rfc3161_timestamp.sql`. Modified files: `RegisterSubmission.js`, `SqlSubmissionRepository.js`, `JsonSubmissionRepository.js`, `index.js`.*

## Comment 1.2 — Insufficient empirical evidence

### Reviewer Comment

The empirical evidence for the reported performance gains (65% cycle time reduction, 45.4% evaluator time reduction) is insufficient. The sample ( $n = 60$  submissions,  $n = 6$  evaluators) is drawn from a single institution, and comparisons with prior workflows rely exclusively on retrospective self-reported administrative survey data, which is susceptible to recall bias. No confidence intervals, formal significance tests (e.g., Mann–Whitney U), or control group comparisons are provided.

**Response:** We agree that the original statistical presentation was inadequate. The revised manuscript now includes:

- 95% confidence intervals for all reported metrics;
- Mann–Whitney  $U$  tests (two-tailed) with exact  $p$ -values: cycle time  $U = 0$ ,  $p = 0.029$ ; evaluator time  $U = 2$ ,  $p = 0.009$ ;
- Effect sizes: rank-biserial  $r = 1.0$  for cycle time, Cohen’s  $d = 1.88$  (large) for evaluator time;
- Fisher’s exact test for dispute rates: 0/60 vs. 14/292,  $p = 0.069$  (one-tailed), explicitly noted as *not reaching conventional significance*;
- Explicit acknowledgment of recall bias, Hawthorne effect, small sample size, and single-institution limitation;
- Hedged language throughout: “preliminary effect-size estimates”, “associated with”, “suggests” instead of causal claims.

We have not added a control group, as this was a real institutional deployment where randomization would have been impractical and ethically problematic. We explicitly state this limitation (Section 6).

**Changes in manuscript:** *Section 5 (Impact) — completely rewritten with statistical tests and hedged language. Section 6 (Limitations) — new “Study design” and “Baseline data quality” subsections.*

## Comment 1.3 — Overstated novelty

### Reviewer Comment

The novelty of the cryptographic contribution is overstated. SHA-256 hashing and HMAC-SHA256 authentication in a Node.js environment represent standard software engineering practice, not a scientific contribution. The claim that no existing platform integrates these mechanisms together requires stronger justification than the comparison presented in Table 3.

**Response:** We fully agree. The individual cryptographic primitives (SHA-256, HMAC-SHA256) are indeed well-established and their implementation does not constitute a scientific novelty claim. The revised manuscript explicitly repositions the contribution:

- A new “Scope of contribution” paragraph in Section 1 states: “*The scientific contribution resides in this systematic integration rather than in the underlying cryptographic primitives.*”
- The abstract now reads: “*The platform combines well-established cryptographic techniques—[...]—into a unified, open-source workflow specifically designed for [...]*”
- All novelty claims have been replaced with “integration engineering” framing throughout.

**Changes in manuscript:** *Abstract, Section 1 (new “Scope of contribution” paragraph), Section 7 (Conclusions) — contribution explicitly framed as integration engineering.*

#### Comment 1.4 — Superficial comparison table

##### Reviewer Comment

The comparative analysis in Table 3 is superficial and appears constructed to favor avalia+Tec. Notably, the paper claims LGPD compliance as a unique advantage while making no assessment of GDPR compliance mechanisms in competing platforms. The comparison criteria should be defined independently of the proposed system’s feature set.

**Response:** We agree that the original binary Yes/No format was reductive and the criteria were not sufficiently independent. The revised Table 3 incorporates:

- A three-level scoring scale (● full/native, ◐ partial/plugin, ✕ absent) to avoid the binary simplification;
- Two new columns not favoring avalia+Tec: “Ext. TSA” (avalia+Tec is rated ◐ partial, as the implementation is new) and “Proven Scale” (avalia+Tec receives ✕, acknowledging its single-institution deployment vs. production systems handling millions of submissions);
- A “GDPR/LGPD” column replacing the LGPD-only column, with footnotes about plugin-based compliance (e.g., OpenReview’s privacy controls);
- An “Ext. Plugins” column explicitly crediting HotCRP and OpenReview for their extension ecosystems;
- Methodology footnotes explaining scoring criteria: “assessed via published documentation, source code inspection, and official feature matrices.”

**Changes in manuscript:** *Table 3 completely redesigned (10 columns, 3-level scale, footnotes, GDPR column, “Proven Scale” column).*

#### Comment 1.5 — Wrong submission category

##### Reviewer Comment

The manuscript has been submitted under the category “Humanities, Arts and Social Sciences,” which is clearly incorrect for a software engineering contribution. The authors should resubmit under an appropriate subject area.

**Response:** We thank the reviewer for noting this administrative error. The resubmission has been reclassified under the “Computer Science” subject area, which is the appropriate category for a software engineering contribution describing a web-based platform with cryptographic components. We apologize for the oversight.

**Changes in manuscript:** *Journal submission metadata corrected to “Computer Science” category.*

## Reviewer #2

### Comment 2.1 — Integration engineering, not scientific novelty

#### Reviewer Comment

The manuscript largely integrates well-established cryptographic primitives (SHA-256, HMAC, RBAC, PostgreSQL RLS) without proposing methodological, algorithmic, or architectural innovations. The contribution appears primarily as an implementation effort rather than a scientifically novel advancement. The authors should clearly articulate what is fundamentally new beyond integration engineering.

**Response:** We agree and have explicitly repositioned the contribution as integration engineering. See our response to Reviewer #1, Comment 1.3. The revised manuscript states that the contribution lies in the *systematic integration* of established primitives into a domain-specific workflow, not in the primitives themselves. We also note that the SoftwareX scope explicitly encompasses “software that has a significant impact on research” regardless of algorithmic novelty.

**Changes in manuscript:** Same as Comment 1.3. Abstract, Section 1 (“Scope of contribution” paragraph), Section 7.

### Comment 2.2 — Insufficient inferential statistics

#### Reviewer Comment

The reported 65% cycle-time reduction and 45.4% evaluator efficiency gain are based on a small, single-institution deployment ( $n = 60$  submissions,  $n = 6$  evaluators). No inferential statistics, confidence intervals, or hypothesis testing are provided. Claims regarding dispute elimination and efficiency improvements are therefore overstated and insufficiently supported.

**Response:** Fully addressed. See our response to Reviewer #1, Comment 1.2. The revised Section 5 now includes 95% CIs, Mann–Whitney  $U$  tests with exact  $p$ -values, effect sizes, and Fisher’s exact test for dispute rates. All claims are hedged appropriately.

**Changes in manuscript:** Same as Comment 1.2. Section 5 (Impact), Section 6 (Limitations).

### Comment 2.3 — Retrospective self-reported data

#### Reviewer Comment

The evaluation relies on retrospective self-reported administrative data. There is no controlled baseline, no randomized comparison, and no longitudinal multi-cohort validation. This weakens the empirical credibility of the reported performance improvements.

**Response:** We acknowledge this limitation explicitly in the revised Section 6 (Limitations), under the new “Baseline data quality” subsection. The text now states: “*Baseline comparisons rely on retrospective self-reported administrative survey data and are subject to recall bias. Evaluators may overestimate time previously spent, inflating the apparent efficiency gain. No objective time-tracking data exists for the pre-deployment period.*” We also note the absence of a randomized comparison and explain why randomization was impractical in a real institutional deployment. Multi-institutional longitudinal validation is identified as planned future work.

**Changes in manuscript:** Section 5 — explicit recall bias and Hawthorne effect warnings. Section 6 — “Baseline data quality” subsection. Section 7 — future work includes multi-institutional deployments with quasi-experimental designs.

## Comment 2.4 — Trusted administrator weakness

### Reviewer Comment

The manuscript explicitly assumes a trusted administrator, which fundamentally weakens the integrity guarantees. Since a malicious root user can rewrite the entire hash chain, the system does not provide strong tamper-resistance in adversarial institutional contexts. Security claims should be toned down accordingly.

**Response:** Addressed via the same RFC 3161 implementation described in our response to Reviewer #1, Comment 1.1. The revised Section 3 now presents a three-class threat model:

1. External (unauthenticated) attacker — mitigated by TLS, rate limiting, input sanitization, JWT auth;
2. Non-privileged insider (authenticated user) — mitigated by RBAC, RLS, HMAC-authenticated forms, hash-chained audit trail;
3. Privileged administrator (root DB access) — mitigated by RFC 3161 external hash anchoring.

The residual limitation is explicitly stated: if the administrator suppresses TSR storage *before* the timestamp is requested, no external anchor exists for that record. This is identified as an area for future improvement (e.g., continuous blockchain anchoring, multi-party key management).

**Changes in manuscript:** *Section 3 — complete rewrite. New code: `TimestampService.js`. All “immutable” → “tamper-evident”.*

## Comment 2.5 — Comparison lacks rigor

### Reviewer Comment

The comparison with platforms such as SIGAA, OpenReview, EasyChair, HotCRP, and SCUPIRA lacks methodological rigor. The functional comparison does not consider plugin capabilities, extensibility, or alternative cryptographic integration possibilities. It presents a binary “Yes/No” simplification that may misrepresent competing systems.

**Response:** Fully addressed. See our response to Reviewer #1, Comment 1.4. The revised Table 3 uses three-level scoring, explicitly credits plugin ecosystems, includes columns where aValia+Tec is rated lower than competitors, and documents the comparison methodology in footnotes.

**Changes in manuscript:** *Same as Comment 1.4. Table 3 redesigned.*

## Comment 2.6 — No formal security validation

### Reviewer Comment

While SHA-256 and HMAC are cited, there is no formal security validation, no penetration testing report, no external audit, and no verification of correct key management practices. Simply calling standard libraries does not guarantee secure deployment.

**Response:** We agree that calling standard libraries does not guarantee secure deployment. The revised manuscript addresses this in two ways:

1. Positive evidence: Section 3 now states that the implementation was checked against the OWASP Top 10 checklist and that `npm audit` reports zero high-severity known vulnerabilities in production dependencies. We also describe current key management practices (JWT secrets and HMAC keys stored as environment variables, never hard-coded).

2. Honest limitation: Section 3 and Section 6 (“Security” subsection) explicitly state: “*No formal penetration test or external security audit has been conducted to date; this represents planned future work.*” and “*These do not constitute a comprehensive security assessment.*”

Formal penetration testing and external audit are listed as future work in Section 7.

**Changes in manuscript:** *Section 3 (Threat Model) — OWASP/npm audit mention, key management description. Section 6 (Limitations) — new “Security” subsection.*

## Comment 2.7 — No stress testing

### Reviewer Comment

Performance testing was conducted on 60 submissions. No stress testing, concurrent user simulation, or high-load benchmarking is presented. It is unclear how the system performs with hundreds or thousands of simultaneous users.

**Response:** We have now conducted and reported load testing. A new load test script (`scripts/load-test.js`) using autocannon (HTTP benchmarking tool for Node.js) was created with three scenarios: GET / (landing page), GET /api/verify/:protocol (integrity verification), and POST /api/inscricao (submission creation with SHA-256 hashing and async RFC 3161 timestamping). Tests were executed against a running avalia+Tec server instance at 100 and 500 concurrent connections for 30 seconds each.

A new Section 4.1 (Load Testing) and Table 5 report the measured results:

- At 100 connections: GET / achieves 1,753 req/s (p50=52 ms, p97.5=98 ms); POST /api/inscricao achieves 1,338 req/s (p50=68 ms, p97.5=133 ms);
- At 500 connections: read operations sustain >1,700 req/s, but POST tail latency increases sharply (p99=8,153 ms) with 1,127 errors out of 47,976 requests, indicating event-loop saturation that confirms the need for horizontal scaling beyond ~500 concurrent writers;
- The text explicitly states that horizontal scaling (load balancing, PostgreSQL read replicas) is a straightforward architectural extension for larger deployments.

The complete raw results (structured JSON with all latency percentiles, throughput, and error counts for all six scenario/concurrency combinations) are included in the replication package (`load-test/load_test_results.json`).

**Changes in manuscript:** *New Section 4.1 (Load Testing), new Table 5 with measured data. New file: `scripts/load-test.js`. Replication package includes raw JSON results.*

## Comment 2.8 — LGPD compliance not demonstrated

### Reviewer Comment

Claims of LGPD compliance are asserted but not rigorously demonstrated through formal compliance mapping, DPIA (Data Protection Impact Assessment), or legal audit references. Compliance statements appear descriptive rather than validated.

**Response:** We acknowledge that full LGPD compliance requires organizational, legal, and technical measures, and a single software platform cannot “be LGPD-compliant” on its own. The revised manuscript:

1. Replaces “LGPD-compliant” with “LGPD-aware design” throughout;
2. Adds a new Table (`tab:lgpdmapping`) that maps specific LGPD articles (Art. 6 I–II, Art. 6 V, Art. 6 VII–VIII, Art. 6 X, Art. 46, Art. 48, Art. 18) to the corresponding technical controls implemented in avalia+Tec;
3. Adds an explicit disclaimer: “*Full LGPD compliance requires institutional policies, a formal RIPD (Data Protection Impact Report), and legal review by each deploying organization. No formal RIPD has been conducted for the current deployment.*”

**Changes in manuscript:** *Section 2 — “LGPLD-aware design” terminology, new Table (tab:lgpdmapping), disclaimer. Section 6 (Limitations) — new “LGPLD compliance” subsection. Section 7 — RIPD listed as future work.*

## Comment 2.9 — No public dataset or replication package

### Reviewer Comment

Although anonymized metrics are said to be “available upon request,” no public dataset or replication package is provided. This limits transparency and reproducibility, which are central to SoftwareX standards.

**Response:** We agree. The revised Data Availability Statement now reads:

*“Anonymized deployment metrics, statistical analysis scripts (Python), and the load-testing framework are available in the replication package at <https://doi.org/10.5281/zenodo.18868994>.”*

The Zenodo replication package is now published at <https://doi.org/10.5281/zenodo.18868994>. The package contains the following evidence files:

- `data/deployment_metrics.csv` — anonymized deployment metrics (no PII);
- `analysis/statistical_analysis.py` — Python script reproducing all CIs, Mann–Whitney *U* tests, effect sizes, and Fisher’s exact test;
- `analysis/output_statistical_analysis.txt` — captured output confirming article values;
- `load-test/load-test.js` — autocannon benchmarking script;
- `load-test/load_test_results.json` — structured JSON with raw results from all six scenario/concurrency combinations.

Machine-readable citation metadata is provided via `CITATION.cff` and `.zenodo.json` in the repository root.

**Changes in manuscript:** *Data Availability Statement updated. Section 6 (Limitations) — “Reproducibility” subsection references Zenodo. Section 7 — replication package mentioned. `CITATION.cff` and `.zenodo.json` added.*

## Comment 2.10 — Verbose manuscript

### Reviewer Comment

The article appears verbose for a SoftwareX original software publication. Sections describing cryptographic fundamentals and workflow diagrams could be condensed. Some conceptual explanations are repetitive and inflate the manuscript without adding technical depth.

**Response:** We have condensed the manuscript by:

- Removing redundant explanations of SHA-256 and HMAC fundamentals (readers of SoftwareX are assumed familiar with standard hashing);
- Consolidating repeated motivational text across sections;
- Tightening the comparison discussion (the redesigned Table 3 conveys more information in less text);
- Moving detailed workflow descriptions to the architecture diagram and its caption;
- Using more concise, direct language throughout.

Despite adding new technical content (RFC 3161 implementation, load testing, LGPLD mapping table, expanded threat model, statistical tests, structured limitations), the overall text is more

focused and less repetitive. We estimate a net reduction in prose volume of approximately 15% (excluding tables and equations).

**Changes in manuscript:** *Throughout the manuscript. Particularly: Section 1 (condensed motivation), Section 2 (removed crypto primer), Section 5 (tables replace verbose descriptions).*

---

## Summary of Changes

| Rev./Pt     | Issue                            | Action Taken                                                                                                                |
|-------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| R1.1 / R2.4 | “Immutable” claim; trusted admin | RFC 3161 TSA implementation; “tamper-evident”; 3-class threat model                                                         |
| R1.2 / R2.2 | No inferential statistics        | 95% CIs, Mann–Whitney $U$ , Fisher’s exact, effect sizes, hedged language                                                   |
| R1.3 / R2.1 | Overstated novelty               | Contribution repositioned as integration engineering                                                                        |
| R1.4 / R2.5 | Superficial comparison           | Table 3 redesigned: 3-level scale, 10 columns, methodology footnotes                                                        |
| R1.5        | Wrong category                   | Reclassified as “Computer Science”                                                                                          |
| R2.3        | Retrospective data               | Recall bias, confounders acknowledged; limitation subsections added                                                         |
| R2.6        | No security validation           | OWASP Top 10 check, npm audit; honest limitation stated                                                                     |
| R2.7        | No stress testing                | Load testing executed: 1,753 req/s GET, 1,338 req/s POST @100 conn; saturation at 500 conn; raw JSON in replication package |
| R2.8        | LGPD not demonstrated            | LGPD mapping table, “LGPD-aware” terminology, RIPD disclaimer                                                               |
| R2.9        | No replication package           | Zenodo replication package published (DOI: 10.5281/zenodo.18868994)                                                         |
| R2.10       | Verbose                          | Text condensed $\sim 15\%$ ; redundancy removed                                                                             |

We trust that these revisions adequately address all concerns raised by both reviewers and the editor. We remain at the reviewers’ disposal for any further clarification.

Sincerely,

L.D.V. Santos and V.R.Q. Leithardt